Name: Chessie Mkashai Nyakoe

Unit: Advanced Software Engineering

Assignment 3

MediConnect tehealth platform implementation using Secure Software Development Lifecycle (SSDLC) framework.

1. STRIDE Threat Model Matrix.

STRIDE is a security threat modelling framework developed by Microsoft to help developers identify and categorize potential security threats to a system. Synthesizes and maps five distinct high-impact threat scenarios affecting the Appointment-Booking and Prescription-Storage modules directly onto specific OWASP Top 10 API Security Risks (2023) classifications.

Spoofing (violates Authenticity): Pretending to someone or something else (logging in as another user).

Tampering (violates Integrity): Modifying data or code maliciously.

Repudiation: claiming you did not perform an action because you can't proof it on the system.

Information Disclosure (violates confidentiality): Exposing of data to unauthorised people.

Elevation of Privilege (violates authorization)

STRIDE Category	Target Module	Threat Scenario Description	Architectural Countermeasure Summary
Tampering	Prescription Storage	Malicious actors alter prescription payload	Cryptographic payload signing via HMAC and strict HMAC
Repudiation	Appointment Booking	Attacker claims they never booked a massive block of slots,	Comprehensive append-only immutable audit logging utilizing asymmetric
Information Disclosure	Prescription Storage	Unauthorized user enumerates sequential prescriptions.	Implement secures UUIDV4 identifiers coupled with rigorous ABAC data policies
Elevation of Privilege	Appointment Booking	A patient alters their JWT claims or updates JSON payload parameters	Cryptographically signed JWT tokens checked against a strict zero-trust middleware layer.
Spoofing	Appointment Booking	Fraudulent 3 party systems send dummy appointment bookings by spoofing upstream.	Mandated authentication alongside Robust API Gateway API key verification layers.

2. Secure Coding Mitigations (TypeScript)

Mitigation 1: Neutralizing Information Disclosure via Secure Object Resolution.

Since I'm using Typescript the code template is:

```
import { Request, Response, NextFunction } from 'express';
import crypto from 'crypto';

const WEBHOOK_SECRET = process.env.PHARMACY_INTEGRATION_SECRET;

export function verifyPayloadIntegrity(req: Request, res: Response, next: NextFun
    const inboundSignature = req.headers['x-mediconnect-signature'] as string;

    if (!inboundSignature || !WEBHOOK_SECRET) {
        return res.status(401).json({ error: "Missing cryptographic verification
    }

    // Compute expected HMAC digest based on raw body stream processing
    const computedSignature = crypto
        .createHmac('sha256', WEBHOOK_SECRET)
        .update(JSON.stringify(req.body))
        .digest('hex');
```

```
    const computedSignature = crypto
        .createHmac('sha256', WEBHOOK_SECRET)
        .update(JSON.stringify(req.body))
        .digest('hex');

    // Use constant-time comparison to completely neutralize timing side-channel
    const isSignatureValid = crypto.timingSafeEqual(
        Buffer.from(inboundSignature, 'hex'),
        Buffer.from(computedSignature, 'hex')
    );

    if (!isSignatureValid) {
        logger.warn("Payload modification attempt detected via invalid signature
        return res.status(403).json({ error: "Payload cryptographic signature mi
    }

    return next();
}
```

3.Secure Code Review Check List

The engineering team must execute, satisfy and log the following structural checks before merging any Pull request (PR) into mainline branches:

1.Authentication and Session Boundaries

Token Integrity: Ensures JWT validation relies solely on asymmetric key rotation patterns rather than arbitrary symmetry schemes.

Transport Security with cookie validation

2.Authorization Controls

Verify that every endpoint implicitly rejects access unless explicit directly role-based or attribute - based or attribute-based authorization hooks.

3.Input Sanitization and Validation Injection Gates

Strict Schemas; Ensures every payload endpoint maps to strict, typed validation schemas that reject unexpected extra parameter keys.

Parameterized queries that confirm object relational mapping layers or raw SQL query blocks rely 100%.

4.Data Privacy and Cryptographic Protection

Validate that all personal health identifiers and prescription strings utilize authenticated encryption befree persisting too underlying block data structures.

Validate that all personal health identifiers (PHI) and prescription strings utilize AES-256-GCM authenticated encryption before persisting to underlying block data structures.

Masking Log Streams: Confirm that log pipelines pass through filter objects to prune access tokens, credit cards, or internal health indicators from plaintext visibility.

5.Availability and Anti-Automation Mitigations

Verify that booking APIs apply isolated Redis-backed slide window rate limits mapped strictly to unique user identifiers combined with client network IP addresses.

References

OWASP Foundation. (2023). OWASP top 10 API security risks – 2023. OWASP API Security Project.

https://owasp.org

Shostack, A. (2014). Threat modeling: Designing for security. John Wiley & Sons.

Vardanega, M., & Node.js Foundation. (2026). Node.js cryptographic API documentation (v24.x).

Node.js Runtime Environment. https://nodejs.org